



CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-[G01] SYSTEMS INTEGRATION AND ARCHITECTURE 1

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project Title: MediBook: Health Appointment & Records
Integration System

Prepared By: Gyle M. Amihan

Date of Submission: June 29, 2026

Version: 1.0

Table of Contents

1. Introduction	3
1.1. Project Title	3
1.2. Project Background	3
1.3. Problem Statement	3
1.4. Project Objectives	3
1.5. Scope of the Project	3
1.6. Limitations of the Project	3
2. Overall Description	3
2.1. Product Perspective	3
2.2. User and Characteristics	3
2.3. Operating Environment	3
2.4. Assumptions and Dependencies	3
3. System Features and Functional Requirements	3
3.1. Feature 1:	4
3.2. Feature 2:	4
4. Non-Functional Requirements	4
5. System Models (Diagrams)	4
5.1. Entity +Relationship Diagrams	4
5.2. Use Case Models	4
5.3. Activity Diagram	4
5.4. Class Diagram	4
5.5. Sequence Diagram	4
6. Appendices	4
7. Initial Project Timeline	5
8. References	5
9. Appendices	5

1. Introduction

1.1. Project Title

MediBook: Health Appointment & Records Integration System

1.2. Project Background

Healthcare facilities, particularly small clinics and barangay health centers in the Philippines, often manage patient appointments and health records manually or through disconnected tools. This results in scheduling conflicts, misplaced records, inefficient patient management, and difficulty tracking patient history.

MediBook addresses these challenges by integrating appointment booking, doctor schedule management, and patient health records into a centralized healthcare management system. Patients can conveniently book appointments through an Android mobile application, while clinic staff manage appointments, doctor schedules, and patient records through a ReactJS web dashboard. Both platforms communicate through a centralized Spring Boot REST API connected to a MySQL database.

1.3. Problem Statement

Many small healthcare facilities continue to rely on manual appointment scheduling and paper-based health records. These traditional methods often result in appointment conflicts, inefficient record management, and delays in patient services. Furthermore, patients have limited access to appointment information and must visit or contact the clinic to schedule consultations.

The absence of an integrated system reduces operational efficiency and increases the likelihood of errors. Therefore, there is a need for a centralized healthcare management system that streamlines appointment scheduling, patient record management, and communication between patients and clinic staff.

1.4. Project Objectives

General Objective

To develop an integrated healthcare management system that centralizes appointment booking, doctor schedule management, and patient health records through a Spring Boot REST API, enabling seamless interaction between an Android mobile application for patients and a ReactJS web application for clinic staff.

Specific Objectives

- Develop a Spring Boot REST API to manage patients, doctors, appointments, and health records.

- Build a ReactJS web application for clinic staff to manage appointments, doctor schedules, and patient records.
- Develop an Android mobile application that allows patients to register, book appointments, and view their appointment schedules.
- Integrate the mobile and web applications using a centralized REST API connected to a shared MySQL database.
- Implement JWT-based authentication to provide secure role-based access for patients, doctors, and clinic staff.

1.5. Scope of the Project

The MediBook system will include the following features:

- Patient registration
- Secure login for patients, doctors, and clinic staff
- Appointment booking through the Android application
- Appointment status tracking (Pending, Confirmed, Completed)
- Doctor schedule management
- Patient health record management
- Web dashboard for clinic staff
- Centralized REST API for communication between all platforms
- JWT-based authentication
- Shared MySQL database for all system data

1.6. Limitations of the Project

The proposed system will not include the following features:

- Online consultation or video conferencing
- Online payment processing
- SMS or email notification services
- Electronic prescription generation
- Pharmacy inventory management
- Laboratory information management
- Hospital billing and insurance claims
- Multi-clinic synchronization
- iOS mobile application support

The initial version is intended for use by a single healthcare facility or clinic.

2. Overall Description

2.1. Product Perspective

MediBook is a client-server healthcare management system composed of three integrated components:

- A Spring Boot backend that provides RESTful API services.
- A ReactJS web application used by clinic staff for appointment and patient management.
- An Android mobile application developed in Kotlin that enables patients to book and monitor appointments.

All components share a centralized MySQL database through the backend API, ensuring consistent and synchronized information across the entire system.

2.2. User and Characteristics

Patient

Patients use the Android mobile application to:

- Register an account
- Log in securely
- Book appointments
- View appointment schedules
- Track appointment status
- View appointment history

Clinic Staff

Clinic staff use the web dashboard to:

- Log in securely
- Manage patient information
- Manage doctor information
- Manage doctor schedules
- Manage appointments
- View appointment queue
- View patient health records

Doctor

Doctors primarily access appointment information prepared by clinic staff. They can:

- Log in securely

- View their appointment queue
- View patient health records
- Create and update health records after consultations

2.3. Operating Environment

Hardware

- Computer for clinic staff
- Android smartphone for patients
- Internet connection
- Cloud-hosted or local server

Software

Backend

- Spring Boot (Java)

Frontend (Web)

- ReactJS

Mobile

- Kotlin

Database

- MySQL

Deployment

- Netlify (Web Application)
- Render (Backend)

API

- REST API using JSON

Authentication

- JWT (JSON Web Token)

2.4. Assumptions and Dependencies

Assumptions

- Users have a stable internet connection.
- Patients own Android smartphones.
- Clinic staff have access to desktop or laptop computers.

- The server remains available during operating hours.

Dependencies

- Spring Boot Framework
- ReactJS
- Android SDK
- MySQL Database
- JWT
- Railway/Render deployment platform
- Netlify hosting service

3. System Features and Functional Requirements

3.1. Feature 1: Appointment Booking

Description:

Allows patients to schedule healthcare appointments through the Android mobile application by selecting an available doctor and appointment schedule.

Functional Requirements:

- Patients shall register and log in before booking appointments.
- The system shall display available doctors and schedules.
- Patients shall submit appointment requests.
- Appointment requests shall initially be marked as **Pending**.
- Clinic staff shall approve or update appointment status.

3.2. Feature 2: Doctor Schedule Management

Description:

Allows clinic staff to create and manage doctors' available schedules through the web dashboard.

Functional Requirements:

- Clinic staff shall create doctor schedules.
- Clinic staff shall update available consultation time slots.
- The system shall prevent patients from booking unavailable schedules.
- Patients shall only view available schedules.

3.3. Feature 3: Patient Health Records

Description:

Allows doctors to create and update patient health records after consultations, while clinic staff may view patient records for administrative purposes.

Functional Requirements:

- Doctors shall create health records after completed consultations.
- Doctors shall update consultation notes and diagnoses.
- The system shall associate every health record with the corresponding appointment.
- Authorized clinic staff may view patient health records.
- Patients shall not modify health records.

3.4. Feature 4: User Authentication

Description:

Provides secure login and role-based access to the system.

Functional Requirements:

- Users shall log in using registered credentials.
- The system shall authenticate users using JWT.
- Patients shall only access patient functions.
- Doctors shall only access doctor functions.
- Clinic staff shall only access administrative functions.
- Unauthorized users shall be denied access.

4. Non-Functional Requirements

Performance

- API responses shall be returned within three seconds under normal usage.
- The system shall support simultaneous access from multiple users without significant performance degradation.

Security

- Passwords shall be encrypted before storage.
- JWT shall be used for authentication.
- Only authenticated users shall access protected resources.
- User roles shall restrict unauthorized access.

Reliability

- The system shall maintain consistent data across web and mobile platforms.
- Database transactions shall preserve data integrity.

Usability

- The mobile application shall provide a simple appointment booking process.
- The web dashboard shall offer an organized interface for clinic staff.
- Navigation shall be intuitive and user-friendly.

Maintainability

- The system shall use a modular architecture.
- Backend APIs shall be reusable by both web and mobile applications.
- Source code shall follow proper coding standards and documentation.

5. System Models (Diagrams)

This section presents the system models that describe the structure, functionality, and interactions of the MediBook: Health Appointment & Records Integration System.

5.1. Entity +Relationship Diagrams

The Entity Relationship Diagram (ERD) illustrates the logical structure of the MediBook database. The system uses a centralized **User** entity for authentication and authorization, while **Patient** and **Doctor** entities store role-specific information. All appointment, scheduling, and health record data are linked through these entities.

Entities:

User

- user_id (PK)
- username
- password
- email
- role

Patient

- patient_id (PK)
- full_name
- date_of_birth
- contact_number
- address

Doctor

- doctor_id (PK)
- full_name
- specialization
- contact_number

ClinicStaff

- staff_id (PK)
- full_name
- contact_number
- position

DoctorSchedule

- schedule_id (PK)
- available_date
- available_time

Appointment

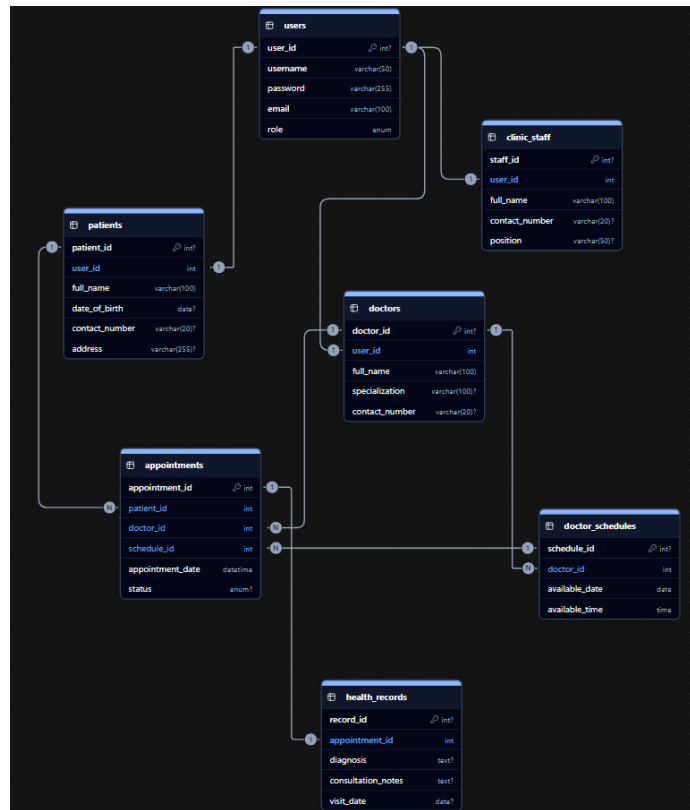
- appointment_id (PK)
- appointment_date
- status

HealthRecord

- record_id (PK)
- diagnosis
- consultation_notes
- visit_date

Relationships:

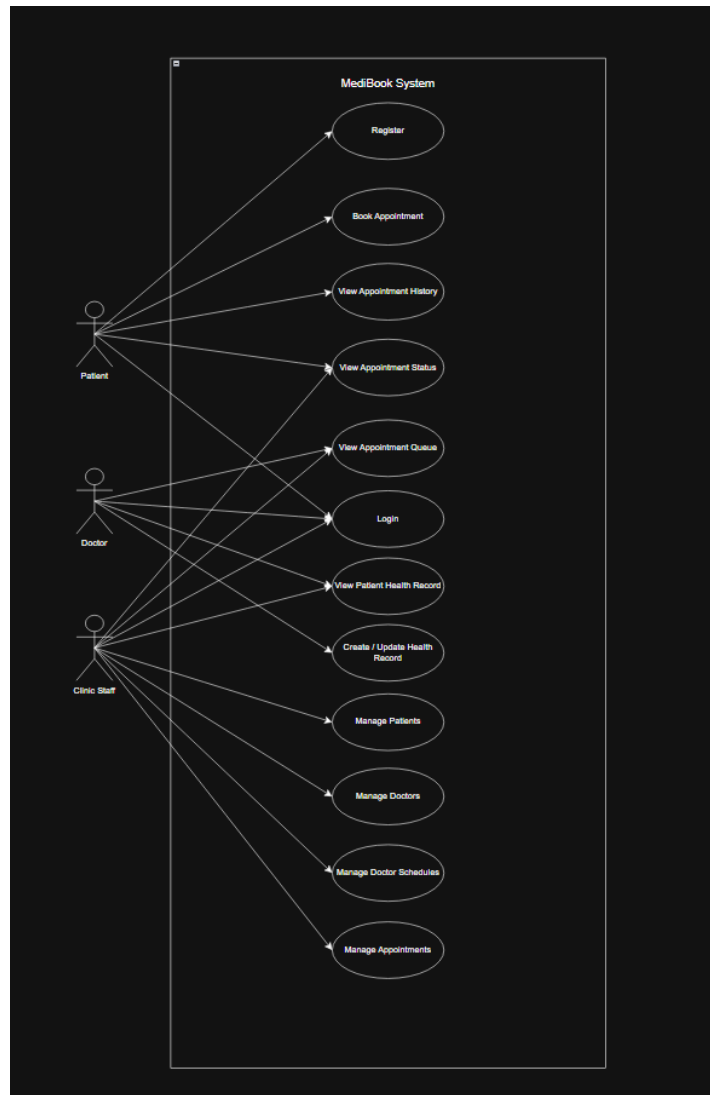
- One **User** may own one **Patient** profile.
- One **User** may own one **Doctor** profile.
- One **User** may own one **Clinic Staff** profile.
- One **Patient** can book many **Appointments**.
- One **Doctor** can handle many **Appointments**.
- One **Doctor** can create many **Doctor Schedules**.
- One **Doctor Schedule** can be assigned to many **Appointments**, provided there are no scheduling conflicts.
- One **Patient** can have many **Health Records**.
- One **Appointment** may generate one **Health Record** after the consultation.



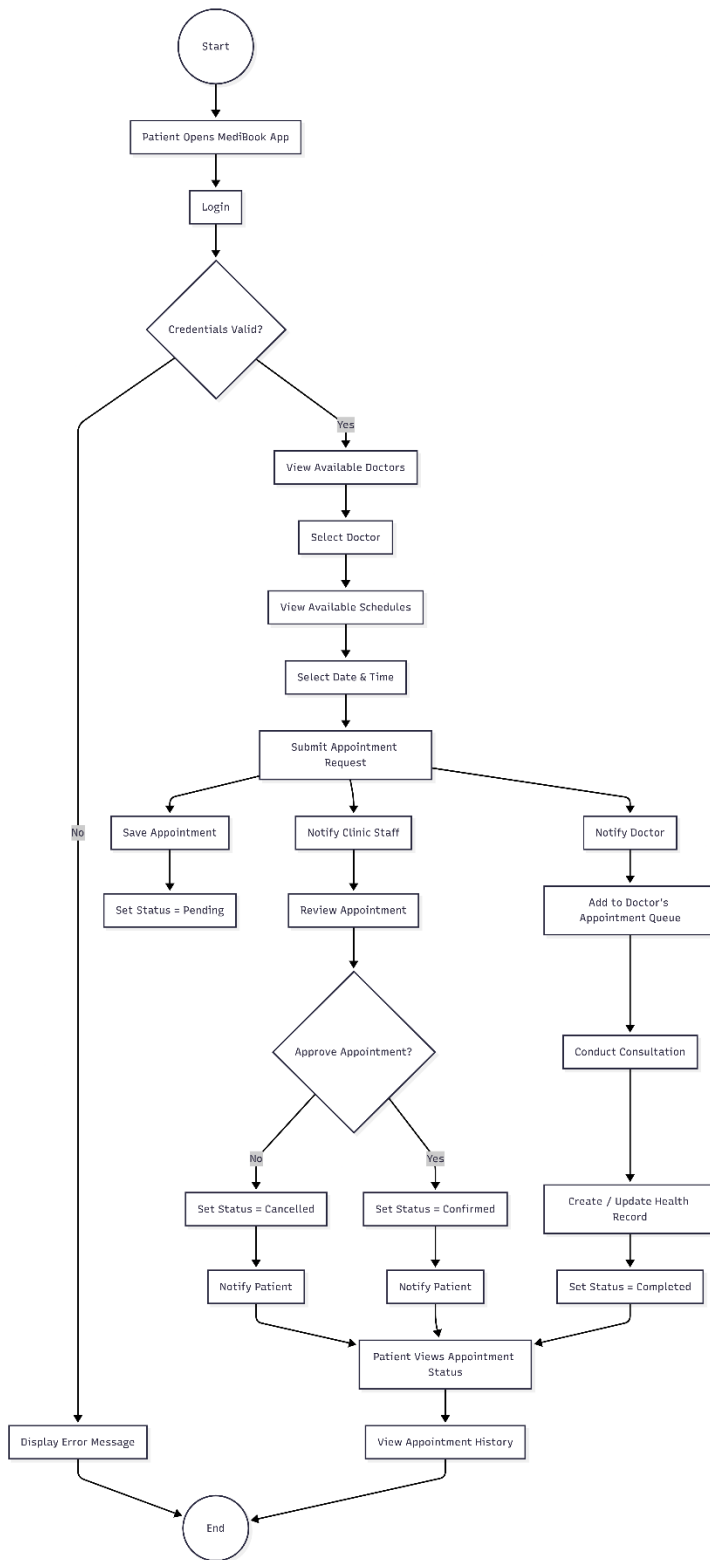
5.2. Use Case Models

Actors

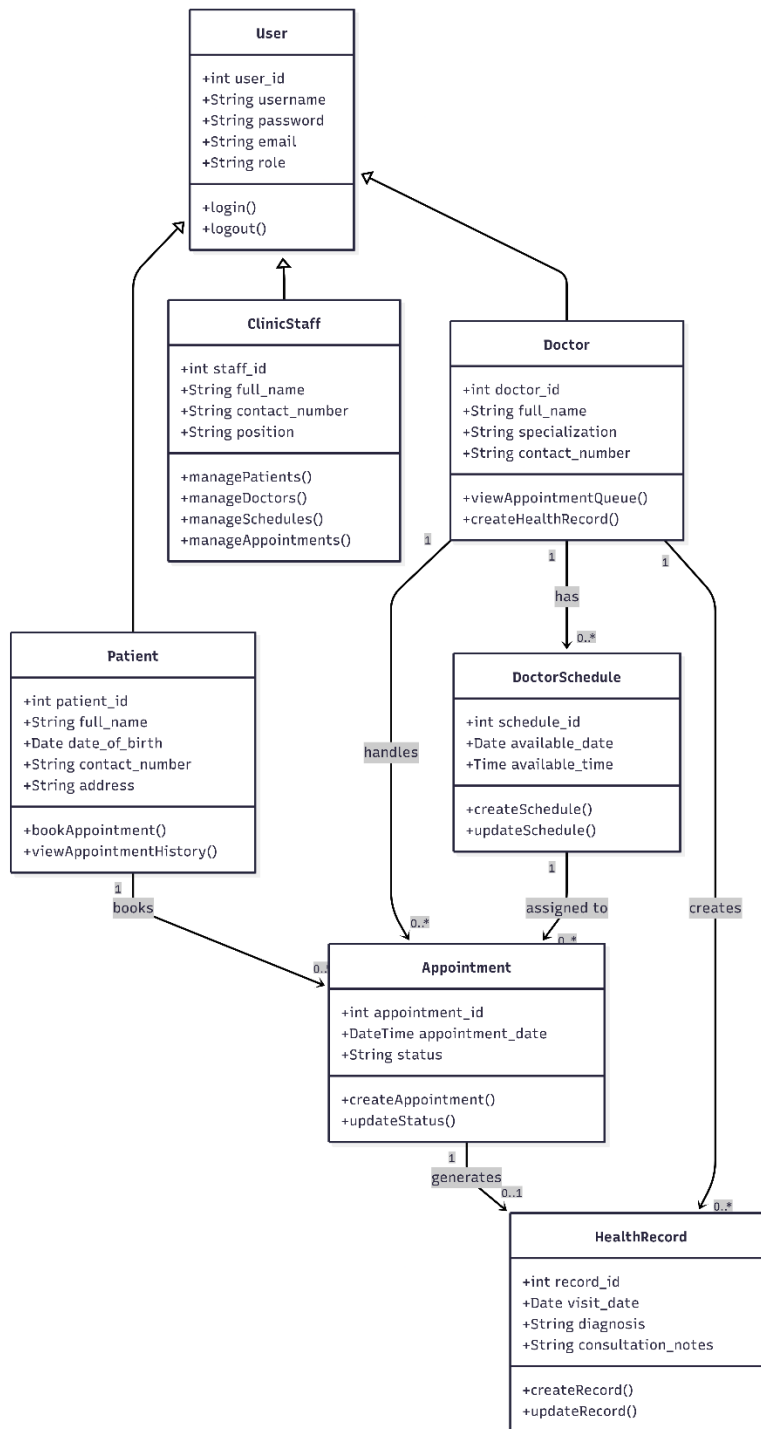
- Patient
- Doctor
- Clinic Staff



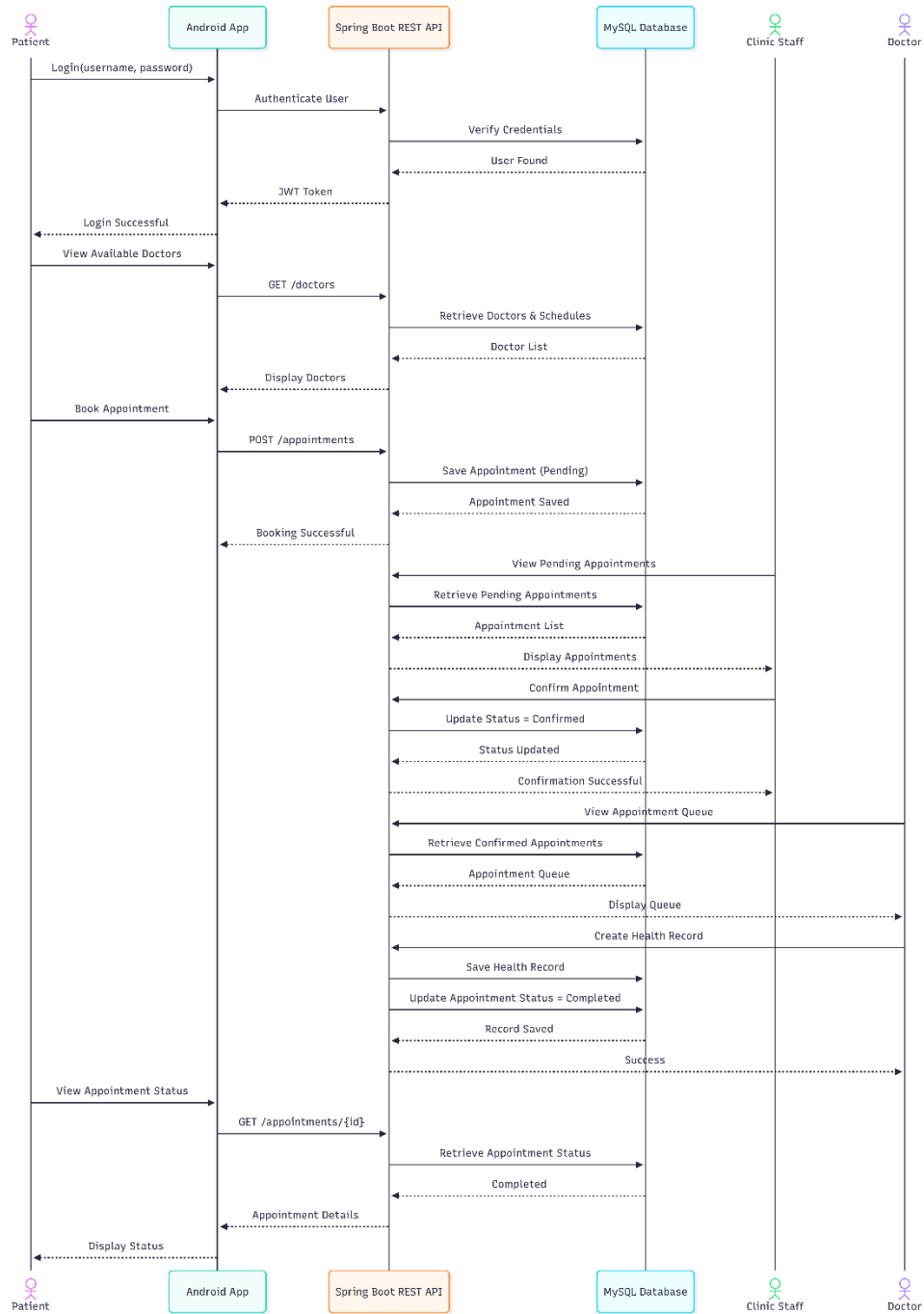
5.3. Activity Diagram



5.4. Class Diagram



5.5. Sequence Diagram



6. Appendices

The following materials support the development and documentation of the MediBook system.

- Project Proposal

- Software Requirements Specification (SRS)
- Entity Relationship Diagram (ERD)
- Use Case Diagram
- Activity Diagram
- Class Diagram
- Sequence Diagram
- Database Schema
-
- API Endpoint Documentation (To Be Finalized)

7. Initial Project Timeline

Week	Planned Activity	Deliverables
Week 1	Project proposal, requirements gathering, and system planning	Approved proposal, finalized SRS, business rules, initial ERD
Week 2	Database design and backend development	MySQL database, Spring Boot project setup, REST API endpoints, JWT authentication
Week 3	Web and mobile application development	ReactJS web dashboard, Android Kotlin mobile application, API integration
Week 4	System integration, testing, deployment, and documentation	Fully integrated system, testing results, deployment, final documentation and presentation

8. References

- Oracle Corporation. (2025). *Java Documentation*. <https://docs.oracle.com/en/java/>
- Spring. (2025). *Spring Boot Documentation*. <https://spring.io/projects/spring-boot>
- React. (2025). *React Documentation*. <https://react.dev/>
- Android Developers. (2025). *Android Developer Documentation*. <https://developer.android.com/>
- MySQL. (2025). *MySQL Documentation*. <https://dev.mysql.com/doc/>
- Mermaid. (2025). *Mermaid Documentation*. <https://mermaid.js.org/>

9. Appendices

- Project Proposal

- Business Rules
- Entity Relationship Diagram (ERD)
- Use Case Diagram
- Activity Diagram
- Class Diagram
- Sequence Diagram
- User Interface Wireframes (*To Be Finalized*)
- Database Schema
- API Endpoint Documentation (*To Be Finalized*)
- Test Cases (*To Be Finalized*)
- Deployment Screenshots (*To Be Finalized*)